Evaluacion 2 (30%)

Guia paso a paso

Proyecto e-commerce Accesorios Chic - frontend-react (Vite + React + TS)

Que falta y como hacerlo, con el porque de cada cosa.

Base: revision del codigo actual en disco (sin cambios). App.tsx con rutas, Productos con filtros, Carrito con checkout inline, AuthContext con destinoTrasLogin, package.json sin script test. Build verificado: tsc OK, vite build 61 modulos OK.

Como usar esta guia: haz los pasos en orden 1 al 7. Cada paso dice QUE hacer, POR QUE (que parte de la rubrica cumple) y COMO verificarlo. Marca el checklist final antes de entregar.

1. Diagnostico: que tienes y que te falta

Ya tienes (no tocar)

- Migracion HTML a React: App.tsx, Layout, Navbar, Footer, Home, Productos, Carrito, Login, Registro, Admin. Estado y props propios.
- Datos simulados + persistencia: src/data/seed.ts + src/storage/useColeccion.ts + claves.ts + localStorage, conectados en Productos.tsx:8 y Carrito.tsx:16-20.
- Bootstrap responsivo: import en main.tsx, clases row/col-md/table-responsive en Productos y Carrito.
- Interactividad base: busqueda ?q= y filtro ?categoria= en Productos.tsx:10-22; checkout con envio/pago en Carrito.tsx:118-304.

Te falta (esto es la guia)

- Rutas verdes Fig.2-4: /categorias, /categoria/:id, /ofertas, /checkout separado, /compra-exitosa, /pago-error. Hoy el checkout esta inline en Carrito.
- Admin Fig.9-10: /ordenes/boleta, /productos/criticos, /reportes, /perfil, /historial-compras.
- CRUD completo: useColeccion.ts solo tiene datos/guardar (replace total). Falta crear/actualizar/eliminar por id.
- Pruebas: 0 archivos *.test.*, sin karma/jest/vitest, sin script test. Es la seccion completa p.11.
- Documentos p.12: ERS V2, documento de cobertura, GitHub publico, zip, presentacion 15+5 min.

2. Paso 1 - Vistas tienda faltantes (Fig.3 a Fig.8)

POR QUE: la rubrica exige 'Vistas adicionales' y el flujo verde de la Figura 3. Sin estas rutas el diagrama queda incompleto y te descuentan navegacion.

Que hacer

- Crea src/pages/Categorias.tsx: grilla de categorias (usa CHIC_KEYS.categorias) que enlace a /productos?categoria={id}. Replica el patron de Home.tsx:220-232.
- Crea src/pages/Ofertas.tsx: misma grilla que Productos.tsx pero filtrando productos con oferta (ej. precioOferta < precio).
- Crea src/pages/Checkout.tsx, CompraExitosa.tsx (Fig.7), PagoError.tsx (Fig.8) con resumen Nombre/Apellidos/Correo + tabla + Total, como hoy hace Carrito.tsx:266-301.
- Registra las 5 rutas en App.tsx junto a las existentes /productos y /carrito.

Como verificar

Entra a /categorias, /ofertas, /checkout, /compra-exitosa y /pago-error. Cada una debe cargar sin error en consola y con layout Navbar/Footer.

3. Paso 2 - Separar el checkout del Carrito

POR QUE: hoy Carrito.tsx:118-304 mezcla carrito + entrega + pago + resumen en un solo componente de 307 lineas. Viola Single Responsibility (p.2) y hace imposible probar el checkout solo.

Que hacer

- Mueve el bloque desde {abierto && (...)} de Carrito.tsx:118 a un componente CheckoutForm con props: entrega, setEntrega, envio, pago, total, onConfirmar.
- Carrito solo lista items, cambia cantidad (81-86), elimina (91-98), vacia y abre checkout. Si hay sesion (useAuth), autorrellena nombre (Carrito.tsx:24-30) como pide la nota p.6.

Como verificar

Vaciar/agregar en Carrito sigue funcionando y el checkout se puede renderizar solo en un test.

4. Paso 3 - Completar panel admin (Fig.9 y Fig.10)

POR QUE: tienes /admin, /admin/productos, /admin/usuarios y sus forms, pero el flujo naranja exige boletas, criticos, reportes y perfil. Sin eso el sistema administrativo queda a medias.

Que hacer

- Crea pages/admin/OrdenesBoleta.tsx (lee CHIC_KEYS.ordenes, muestra folio + detalle).
- Crea pages/admin/ProductosCriticos.tsx (filtra stock <= stockCritico).
- Crea pages/admin/Reportes.tsx (conteos: compras, productos, usuarios) y Perfil.tsx.
- Agrega links en components/admin/AdminLayout y protege todo con RutaProtegida roles={[1,2]} como ya hace App.tsx:52-64.

Como verificar

Como ADMIN entra a cada ruta /admin/* sin 403; como CUSTOMER debe dar vista sin permisos.

5. Paso 4 - CRUD completo (requisito p.1)

POR QUE: useColeccion.ts:10-13 solo reemplaza el arreglo completo. La rubrica pide Crear/Leer/Actualizar/Eliminar sobre el archivo JS. Sin funciones por id no puedes demostrar CRUD en la defensa.

Que hacer

- En useColeccion.ts agrega: crear(item), actualizarPorId(id, cambios), eliminarPorId(id), cada una leyendo localStorage, modificando y llamando guardar.
- Usa esas funciones en AdminProductoForm, AdminUsuarioForm y en la nueva vista de categorias. Deja seed.ts solo como datos iniciales.

Como verificar

Crea un producto, editalo, borralo y recarga: debe persistir en localStorage (Application > Local Storage en F12).

6. Paso 5 - Pruebas (seccion p.11, la mas pesada)

POR QUE: hoy hay 0 tests (glob no encontro *.test.* ni karma/jest config). La propuesta del docente exige 5 tipos: renderizado, props, estado y eventos. Sin esto no hay documento de cobertura.

Que hacer

- Instala en frontend-react: vitest jsdom @testing-library/react @testing-library/jest-dom @testing-library/user-event. Vitest usa la misma sintaxis test/expect que Jasmine/Jest de la guia pero funciona con Vite+TS (Karma necesita Chrome y rompe en EC2 t2.micro).
- Crea vitest.config.ts (environment jsdom, setupFiles src/setupTests.ts, maxWorkers 2 por t2.micro) y script test: vitest run.
- Escribe 5 tests con tus archivos reales: 1) Productos renderiza tarjetas; 2) Carrito vacio muestra 'Tu carrito esta vacio'

(Carrito.tsx:38); 3) TarjetaProducto recibe prop producto; 4) escribir en input Productos.tsx:32-39 filtra; 5) click + en Carrito.tsx:81-86 cambia cantidad.

Como verificar

Corre npm test: todo en verde. Corre npm test -- --coverage: se genera coverage/. Guarda captura y el % por archivo.

7. Paso 6 - Cobertura, build y entrega (p.12)

POR QUE: entregas 4 archivos + presentacion. Sin ERS V2 y sin cobertura no hay nota aunque el codigo funcione.

Que hacer

- Cobertura: npm test -- --coverage. Redacta 1 pagina por cada uno de los 5 tests: que verifica, dato de entrada, resultado esperado, % cubierto.
- Build: npm run build (tsc -b + vite build). Debe decir built sin errores. Tu salida es dist/, no build/ (la guia es de CRA).
- ERS V2: actualiza requerimientos, herramientas (Vite, Vitest, Bootstrap), propuesta y mapa de rutas Fig.2/Fig.10 con lo que hiciste en pasos 1-4.
- GitHub: repo publico solo frontend, rama main limpia, commits por integrante (el docente revisa cambios en GitHub). Sube tambien el zip del proyecto frontend.
- Presentacion 15+5: demo en vivo Home > Productos con filtro > Detalle > Carrito > Checkout > Exitosa/Error + Admin criticos/reportes; luego npm test en verde y reporte de cobertura; cierra con ronda de preguntas (cada uno explica su parte y sus commits).

Equivalencias Vite vs guia (para no confundirte)

- npm start / puerto 3000 / carpeta build / Node 14 => npm run dev / puerto 5173 / carpeta dist / Node 22.
- En EC2 abre el puerto 5173 TCP y prueba curl http://tu-ip-publica:5173; para demo usa npm run dev -- --host --port 5173 o npm run preview tras el build.

Checklist final antes de entregar

- [ ] /categorias, /ofertas, /checkout, /compra-exitosa, /pago-error navegan sin consola roja.
- [ ] Checkout separado de Carrito y autorrellena con sesion.
- [ ] Admin abre boleta, criticos, reportes y perfil con roles 1-2.
- [ ] CRUD por id funciona y persiste tras recargar.
- [ ] npm test en verde con 5 tipos de prueba y coverage generado.
- [ ] npm run build OK y dist/ generado.
- [ ] GitHub publico + zip + ERS V2 + cobertura listos; demo 15 min ensayada.